

Date: 21/07/2026

EXP:1 Mean and variance of a discrete distribution

Aim:

To find mean and variance of arrival of objects from the feeder using probability distribution

Software Required:

Python and Visual components tool

Theory:

The expectation or the mean of a discrete random variable is a weighted average of all possible values of the random variable. The weights are the probabilities associated with the corresponding values. It is calculated as,

$$E(X) = \mu = \sum_i x_i p_i \quad i = 1, 2, \dots, n$$

$$E(X) = x_1 p_1 + x_2 p_2 + \dots + x_n p_n.$$

The variance of a random variable shows the variability or the scatterings of the random variables. It shows the distance of a random variable from its mean. It is calculated as

$$\sigma_x^2 = \text{Var}(X) = \sum_i (x_i - \mu)^2 p(x_i) = E(X - \mu)^2 \text{ or, } \text{Var}(X) = E(X^2) - [E(X)]^2.$$

$$E(X^2) = \sum_i x_i^2 p(x_i), \text{ and } [E(X)]^2 = [\sum_i x_i p(x_i)]^2 = \mu^2.$$

Procedure :

1. Construct frequency distribution for the data
2. Find the probability distribution from frequency distribution.
3. Calculate mean using

$$E(X) = \mu = \sum_i x_i p_i \quad i = 1, 2, \dots, n$$

4. Find

$$E(X^2) = \sum_i x_i^2 p(x_i).$$

5. Calculate variance using

$$\text{Var}(X) = E(X^2) - [E(X)]^2$$

Program:

Name: Tanushree G

Register No: 212225040462

```
import numpy as np
L=[int(i) for i in input().split()]
N=len(L); M=max(L)
x=list();f=list()
for i in range (M+1):
    c = 0
    for j in range(N):
        if L[j]==i:
            c=c+1
    f.append(c)
    x.append(i)
sf=np.sum(f)
p=list()
for i in range(M+1):
    p.append(f[i]/sf)
mean=np.inner(x,p)
EX2=np.inner(np.square(x),p)
var=EX2-mean**2
SD=np.sqrt(var)
print("The Mean arrival rate is %.3f "%mean)
print("The Variance of arrival from feeder is %.3f "%var)
print("The Standard deviation of arrival from feeder is %.3F "%SD)
```

Output :

```
27 32 12 26 29 19
The Mean arrival rate is 24.167
The Variance of arrival from feeder is 45.139
The Standard deviation of arrival from feeder is 6.719
```

Results:

The mean and variance of arrivals of objects from feeder using probability distribution are calculated.

Date: 28/07/2026

EXP:2 Fitting Poisson distribution

Aim:

To fit Poisson distribution for the arrival of objects per minute from the feeder

Software required:

Python and Visual component tool

Theory:

The Poisson distribution is the discrete probability distribution of the number of events occurring in a given time period, given the average number of times the event occurs over that time period.

If λ is mean, then the probability mass function of Poisson distribution is

$$P(X = x) = e^{-\lambda} \frac{\lambda^x}{x!}, \quad x = 0, 1, 2, \dots$$

1. An event can occur any number of times during a time period.
2. Events occur independently.
3. The rate of occurrence is constant.
4. The probability of an event occurring is proportional to the length of the time period.

Procedure :

1. *Compute mean* $= \frac{\sum fx}{N}$, $N = \sum f$.
2. *Calculate the expected frequencies from the probability mass function*
$$P(X = x) = e^{-\lambda} \frac{\lambda^x}{x!}, \quad x = 0, 1, 2, \dots$$
3. *Calculate the expected frequencies*
$$N \times P(X = x), \quad x = 0, 1, 2, \dots, n$$
4. *Calculate* $\chi^2 = \sum \frac{(O-E)^2}{E}$, *where o and E are observed and expected frequencies*
5. *Test* χ^2 *at 1% level of significance and write the conclusion*

Program:

Name: Tanushree G
Register No: 212225040462

```
import numpy as np
import math
import scipy.stats
```

```

L=[int(i) for i in input().split()]
N=len(L); M=max(L)
X=list();f=list()
for i in range (M+1):
    c = 0
    for j in range(N):
        if L[j]==i:
            c=c+1
    f.append(c)
    X.append(i)
sf=np.sum(f)
p=list()
for i in range(M+1):
    p.append(f[i]/sf)
mean=np.inner(X,p)
p=list();E=list();xi=list()
print("X P(X=x) Obs.Fr Exp.Fr xi")
print("-----")
for x in range(M+1):
    p.append(math.exp(-mean)*mean**x/math.factorial(x))
    E.append(p[x]*sf)
    xi.append((f[x]-E[x])**2/E[x])
    print("%2.2f %2.3f %4.2f %3.2f %3.2f"%(x,p[x],f[x],E[x],xi[x]))
print("-----")
cal_chi2_sq=np.sum(xi)
print("Calculated value of Chi square is %4.2f"%cal_chi2_sq)
table_chi2=scipy.stats.chi2.ppf(1-.01,df=M)
print("Table value of chi square at 1 level is %4.2f"%table_chi2)
if cal_chi2_sq<table_chi2:
    print("The given data can be fitted in poisson Distribution at 1% LOS")
else:
    print("The given data cannot be fitted in Poisson Distribution at 1% LOS")

```

Output :

```

9 7 1 2 0 8 5
X P(X=x) Obs.Fr Exp.Fr xi
-----
0.00 0.010 1.00 0.07 11.88
1.00 0.047 1.00 0.33 1.35
2.00 0.108 1.00 0.76 0.08
3.00 0.165 0.00 1.15 1.15
4.00 0.188 0.00 1.32 1.32
5.00 0.172 1.00 1.20 0.03
6.00 0.131 0.00 0.92 0.92
7.00 0.086 1.00 0.60 0.27
8.00 0.049 1.00 0.34 1.26
9.00 0.025 1.00 0.17 3.92
-----
Calculated value of Chi square is 22.19
Table value of chi square at 1 level is 21.67
The given data cannot be fitted in Poisson Distribution at 1% LOS

```

Results:

The Poisson distribution is fitted for the objects arrived from feeder per minute and the data is tested using Chi-square test.

Date: 04/08/2026

EXP 3: Correlation and regression for data analysis

Aim:

To analyse given data using coefficient of correlation and regression

line

X	25	28	35	32	31	36	29	38	34	32
Y	43	46	49	41	36	32	31	30	33	39

Software required:

Python

Theory:

Correlation describes the strength of an association between two variables, and is completely symmetrical, the correlation between A and B is the same as the correlation between B and A. However, if the two variables are related it means that when one changes by a certain amount the other changes on an average by a certain amount.

If y represents the dependent variable and x the independent variable, this relationship is described as the regression of y on x. The relationship can be represented by a simple equation called the regression equation. The regression equation representing how much y changes with any given change of x can be used to construct a regression line on a scatter diagram, and in the simplest case this is assumed to be a straight line.

Procedure:

1. Compute $\sum X, \sum Y, \sum X^2, \sum Y^2$ and $\sum XY$.
2. Calculate correlation coefficient by

$$\rho = \frac{N \sum XY - \sum X \sum Y}{\sqrt{N \sum X^2 - (\sum X)^2} \sqrt{N \sum Y^2 - (\sum Y)^2}}$$

3. Compute $\bar{X} = \frac{\sum X}{N}$ and $\bar{Y} = \frac{\sum Y}{N}$

4. Calculate regression coefficient by

$$b_{YX} = \frac{N \sum XY - \sum X \sum Y}{N \sum X^2 - (\sum X)^2}$$

5. The regression line Y on X is given by

$$Y = b_{YX}(X - \bar{X}) + \bar{Y}$$

6. Plot the given data and the Regression line in a graph.

PROGRAM:

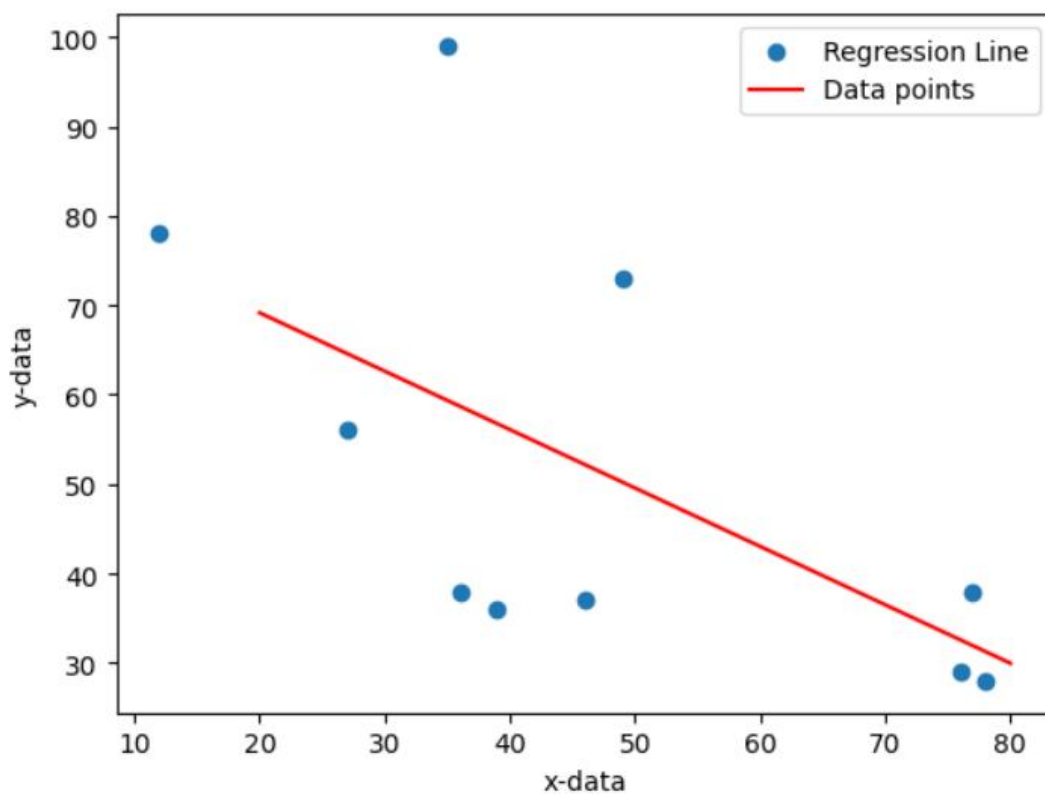
Name: Tanushree G

Register No: 212225040462

```
import numpy as np
import math
import matplotlib.pyplot as plt
x=[ int(i) for i in input().split()]
y=[ int(i) for i in input().split()]
N=len(x)
Sx=0
Sy=0
Sxy=0
Sx2=0
Sy2=0
for i in range(0,N):
    Sx=Sx+x[i]
    Sy=Sy+y[i]
    Sxy=Sxy+x[i]*y[i]
    Sx2=Sx2+x[i]**2
    Sy2=Sy2+y[i]**2
r=(N*Sxy-Sx*Sy)/(math.sqrt(N*Sx2-Sx**2)*math.sqrt(N*Sy2-Sy**2))
print("The Correlation coefficient is %0.3f"%r)
byx=(N*Sxy-Sx*Sy)/(N*Sx2-Sx**2)
xmean=Sx/N
ymean=Sy/N
print("The Regression line Y on X is ::: y = %0.3f + %0.3f (x-%0.3f)"%(ymean,byx,xmean))
plt.scatter(x,y)
def Reg(x):
    return ymean + byx*(x-xmean)
x=np.linspace(20,80,51)
y1=Reg(x)
plt.plot(x,y1,'r')
plt.xlabel('x-data')
plt.ylabel('y-data')
plt.legend(['Regression Line','Data points'])
```

Output :

```
35 12 27 49 78 39 77 36 76 46
99 78 56 73 28 36 38 38 29 37
The Correlation coefficient is -0.611
The Regression line Y on X is :::  $y = 51.200 + -0.653 (x-47.500)$ 
<matplotlib.legend.Legend at 0x2426b4767d0>
```



RESULT:

The Correlation and regression for data analysis of objects from feeder using probability distribution are calculated.

Date: 08/08/2026

EXP 4: Single server with infinite capacity(M/M/1):(∞/FIFO)

Aim:

To find (a) average number of materials in the system (b) average number of materials in the conveyor (c) waiting time of each material in the system (d) waiting time of each material in the conveyor, if the arrival of materials follow Poisson process with the mean interval time 12 seconds, service time of lathe machine follows exponential distribution with mean service time 1 second and average service time of robot is 7seconds.

Software required:

Visual components and Python

Theory:

Queuing are the most frequently encountered problems in everyday life. For example, queue at a cafeteria, library, bank, etc. Common to all of these cases are the arrivals of objects requiring service and the attendant delays when the service mechanism is busy. Waiting lines cannot be eliminated completely, but suitable techniques can be used to reduce the waiting time of an object in the system. A long waiting line may result in loss of customers to an organization. Waiting time can be reduced by providing additional service facilities, but it may result in an increase in the idle time of the service mechanism.

This is a queuing model in which the arrival is Markovian and departure distribution is also Markovian, number of server is one and size of the queue is also Markovian, no. of server is one and size of the queue is infinite and service discipline is 1st come 1st server (FCFS) and the calling source is also finite.

Procedure :

1. Probability of zero units in the queue (P_0) = $1 - \frac{\lambda}{\mu}$
2. Average queue length (L_q) = $\frac{\lambda^2}{\lambda(\lambda - \mu)}$
3. Average number of units in the system (L_s) = $\frac{\mu}{\lambda - \mu}$
4. Average waiting time of an arrival (W_q) = $\frac{\lambda}{\mu(\lambda - \mu)}$
5. Average waiting time of an arrival in the system (W_s) = $\frac{1}{\lambda - \mu}$

Program:

Name: Tanushree G

Register No: 212225040462

```
arr_time=float(input("Enter the mean inter arrival time of objects from Feeder (in secs): "))
ser_time=float(input("Enter the mean inter service time of Lathe Machine (in secs) : "))
Robot_time=float(input("Enter the Additional time taken for the Robot (in secs) : "))
lam=1/arr_time
mu=1/(ser_time+Robot_time)
print("-----")
print("Single Server with Infinite Capacity - (M/M/1):(oo/FIFO)")
print("-----")
print("The mean arrival rate per second : %0.2f "%lam)
print("The mean service rate per second : %0.2f "%mu)
if (lam < mu):
    Ls=lam/(mu-lam)
    Lq=Ls-lam/mu
    Ws=Ls/lam
    Wq=Lq/lam
    print("Average number of objects in the system : %0.2f "%Ls)
    print("Average number of objects in the conveyor : %0.2f "%Lq)
    print("Average waiting time of an object in the system : %0.2f secs"%Ws)
    print("Average waiting time of an object in the conveyor : %0.2f secs"%Wq)
    print("Probability that the system is busy : %0.2f "%(lam/mu) )
    print("Probability that the system is empty : %0.2f "%(1-lam/mu) )
else:
    print("Warning! Objects Over flow will happen in the conveyor")
print("-----")
```

Output:

```
Enter the mean inter arrival time of objects from Feeder (in secs): 10
Enter the mean inter service time of Lathe Machine (in secs) : 2
Enter the Additional time taken for the Robot (in secs) : 9
-----
Single Server with Infinite Capacity - (M/M/1):(oo/FIFO)
-----
The mean arrival rate per second : 0.10
The mean service rate per second : 0.09
Warning! Objects Over flow will happen in the conveyor
-----
```

Result:

The average number of material in the system and in the conveyor and waiting time are successfully found.

Date: 13/08/2026

EXP 5: Multiple server with infinite capacity - (M/M/c):(∞/FIFO)

\Aim:

To find (a) average number of materials in the system (b) average number of materials in the conveyor (c) waiting time of each material in the system (d) waiting time of each material in the conveyor, if the arrival of materials follow Poisson process with the mean interval time 10 seconds, service time of two lathe machine follow exponential distribution with mean service time 1 second and average service time of robot is 7seconds.

Software required:

Visual components and Python

Theory:

Queuing are the most frequently encountered problems in everyday life. For example, queue at a cafeteria, library, bank, etc. Common to all of these cases are the arrivals of objects requiring service and the attendant delays when the service mechanism is busy. Waiting lines cannot be eliminated completely, but suitable techniques can be used to reduce the waiting time of an object in the system. A long waiting line may result in loss of customers to an organization. Waiting time can be reduced by providing additional service facilities, but it may result in an increase in the idle time of the service mechanism.

Procedure:

1. Traffic intensity $\rho = \frac{\lambda}{c\mu}$
2. Average number of objects in the queue $L_q = \frac{1}{c!} \frac{1}{c} \left(\frac{\lambda}{\mu}\right)^{c+1} \frac{1}{(1-\rho)^2} P_0$,

$$\text{where } P_0 = \left[\sum_{n=0}^{c-1} \frac{1}{n!} \left(\frac{\lambda}{\mu}\right)^n + \frac{1}{c!} \left(\frac{\lambda}{\mu}\right)^c \frac{1}{1-\rho} \right]^{-1}$$

3. Average number of objects in the system $L_s = L_q + \frac{\lambda}{\mu}$
4. Average waiting time in the system $W_s = \frac{L_s}{\lambda}$
5. Average waiting time in the queue $W_q = \frac{L_q}{\lambda}$

PROGRAM:

Name: Tanushree G

Register No: 212225040462

```
import math
arr_time=float(input("Enter the mean inter arrival time of objects from Feeder (in secs): "))
ser_time=float(input("Enter the mean inter service time of Lathe Machine (in secs) : "))
Robot_time=float(input("Enter the Additional time taken for the Robot (in secs) : "))
c=int(input("Number of service centre : "))
lam=1/arr_time
mu=1/(ser_time+Robot_time)
print("-----")
print("Multiple Server with Infinite Capacity - (M/M/c):(oo/FIFO)")
print("-----")
print("The mean arrival rate per second : %.2f "%lam)
print("The mean service rate per second : %.2f "%mu)
rho=lam/(c*mu)
sum=(lam/mu)**c*(1/(1-rho))/math.factorial(c)
for i in range(0,c):
    sum=sum+(lam/mu)**i/math.factorial(i)
P0=1/sum
if (rho<1):
    Lq=(P0/math.factorial(c))*(1/c)*(lam/mu)**(c+1)/(1-rho)**2
    Ls=Lq+lam/mu
    Ws=Ls/lam
    Wq=Lq/lam
    print("Average number of objects in the system : %.2f "%Ls)
    print("Average number of objects in the conveyor : %.2f "%Lq)
    print("Average waiting time of an object in the system : %.2f secs"%Ws)
    print("Average waiting time of an object in the conveyor : %.2f secs"%Wq)
    print("Probability that the system is busy : %.2f "%(rho))
    print("Probability that the system is empty : %.2f "%(1-rho))
else:
    print("Warning! Objects Over flow will happen in the conveyor")
print("-----")
```

OUTPUT:

```
Enter the mean inter arrival time of objects from Feeder (in secs): 15
Enter the mean inter service time of Lathe Machine (in secs) : 3
Enter the Additional time taken for the Robot (in secs) : 8
Number of service centre : 3
-----
Multiple Server with Infinite Capacity - (M/M/c):(oo/FIFO)
-----
The mean arrival rate per second : 0.07
The mean service rate per second : 0.09
Average number of objects in the system : 0.75
Average number of objects in the conveyor : 0.01
Average waiting time of an object in the system : 11.20 secs
Average waiting time of an object in the conveyor : 0.20 secs
Probability that the system is busy : 0.24
Probability that the system is empty : 0.76
-----
```

Result:

Thus, the average number of materials in the system and conveyor, waiting time of each material in the system and conveyor is found successfully.

Date: 19/08/2026

EXP 6: Series Queues with infinite capacity - Open Jackson Network

Aim:

To find (a) average number of materials in the system (b) average number of materials in the each conveyor of (c) waiting time of each material in the system (d) waiting time of each material in each conveyor, if the arrival of materials follow Poisson process with the mean interval time 12 seconds, service time of lathe machine in series follow exponential distribution with service time 1 second, 1.5 seconds and 1.3 seconds respectively and average service time of robot is 7 seconds.

Software required:

Visual components and Python

Theory

Open Jackson Networks

An open Jackson network is a system of k service stations where station i ($i = 1, 2, 3, \dots, k$) has the following characteristics.

- (i) An infinite queue capacity
- (ii) Customer arrive at station i from outside of the system according to a Poisson processes with parameter r_i .
- (iii) C_i servers at station i with an exponential service time distribution with parameter μ_i
- (iv) Customers completing service at station i next go to station j .
- (v) Let λ_j denote the total arrival rate of customers to the station S_j . Then the traffic flow equation is

$$\lambda_j = r_j + \sum_{i=1}^m \lambda_i p_{ij}$$

Procedure:

1. Average number of customers in the system S_j is $L_{S_j} = \frac{\lambda_j}{\lambda_j - \mu_j}$
2. Average number of customers in the overall system $L_S = \sum_{j=1}^k L_{S_j}$
3. Average waiting time in the system $W_S = \frac{L_S}{r_1 + r_2 + \dots + r_k}$

PROGRAM:

Name: Tanushree G

Register No: 212225040462

```
arr_time=float(input("Enter the mean inter arrival time of objects from Feeder (in secs): "))
ser_time1=float(input("Enter the mean inter service time of Lathe Machine 1 (in secs) : "))
ser_time2=float(input("Enter the mean inter service time of Lathe Machine 2 (in secs) : "))
ser_time3=float(input("Enter the mean inter service time of Lathe Machine 3 (in secs) : "))
Robot_time=float(input("Enter the Additional time taken for the Robot (in secs) : "))
lam=1/arr_time
mu1=1/(ser_time1+Robot_time)
mu2=1/(ser_time2+Robot_time)
mu3=1/(ser_time3+Robot_time)
print("-----")
print("Series Queues with infinite capacity- Open Jackson Network")
print("-----")
if (lam < mu1) and (lam < mu2) and (lam < mu3):
    Ls1=lam/(mu1-lam)
    Ls2=lam/(mu2-lam)
    Ls3=lam/(mu3-lam)
    Ls=Ls1+Ls2+Ls3
    Lq1=Ls1-lam/mu1
    Lq2=Ls2-lam/mu2
    Lq3=Ls3-lam/mu3
    Wq1=Lq1/lam
    Wq2=Lq2/lam
    Wq3=Lq3/lam
    Ws=Ls/(3*lam)
    print("Average number of objects in the system S1 : %0.2f "%Ls1)
    print("Average number of objects in the system S2 : %0.2f "%Ls2)
    print("Average number of objects in the system S3 : %0.2f "%Ls3)
    print("Average number of objects in the overall system      : %0.2f "%Ls)
    print("Average number of objects in the conveyor S1  : %0.2f "%Lq1)
    print("Average number of objects in the conveyor S2  : %0.2f "%Lq2)
    print("Average number of objects in the conveyor S3  : %0.2f "%Lq3)
    print("Average waiting time of an object in the conveyor S1 : %0.2f secs"%Wq1)
    print("Average waiting time of an object in the conveyor S2 : %0.2f secs"%Wq2)
    print("Average waiting time of an object in the conveyor S3 : %0.2f secs"%Wq3)
else:
    print("Warning! Objects Over flow will happen in the conveyor")
print("-----")
```

OUTPUT:


```
Enter the mean inter arrival time of objects from Feeder (in secs): 19
Enter the mean inter service time of Lathe Machine 1 (in secs) : 4
Enter the mean inter service time of Lathe Machine 2 (in secs) : 3
Enter the mean inter service time of Lathe Machine 3 (in secs) : 2
Enter the Additional time taken for the Robot (in secs) : 9
```

```
-----
Series Queues with infinite capacity- Open Jackson Network
```

```
-----
Average number of objects in the system S1 : 2.17
Average number of objects in the system S2 : 1.71
Average number of objects in the system S3 : 1.37
Average number of objects in the overall system : 5.26
Average number of objects in the conveyor S1 : 1.48
Average number of objects in the conveyor S2 : 1.08
Average number of objects in the conveyor S3 : 0.80
Average waiting time of an object in the conveyor S1 : 28.17 secs
Average waiting time of an object in the conveyor S2 : 20.57 secs
Average waiting time of an object in the conveyor S3 : 15.12 secs
-----
```

Result

The average number of material in the system and in the conveyor and waiting time are successfully found.

